Operating Systems — ★★★Exam Answers (Chapters 6–9)

Muhtasim Sir's Part — Concise Exam Format

Based on: *Operating System Concepts, 10th Edition*

CHAPTER 6: SYNCHRONIZATION TOOLS

Q1. What causes data inconsistency? What is the critical-section problem?

★★★[17-18, 18-19, 19-20, 20-21 FINAL]

Data Inconsistency — Race Condition

Race condition occurs when multiple processes access shared data concurrently and at least one modifies it. The final result depends on the **execution order** (non-deterministic).

Example: Counter increment/decrement

```
counter++;  // Expands to: R1=counter; R1=R1+1; counter=R1
counter--;  // Expands to: R2=counter; R2=R2-1; counter=R2
```

If interleaved (counter=5 initially):

```
T1: R1 = counter     (R1=5)
T2: R2 = counter     (R2=5)
T3: R1 = R1 + 1      (R1=6)
T4: R2 = R2 - 1      (R2=4)
T5: counter = R1     (counter=6)
T6: counter = R2     (counter=4)  // Lost update!
```

Expected: 5+1-1=5, Actual: 4 → **Data inconsistency**

Critical-Section Problem

Critical section = code segment where shared data is accessed.

Problem: Design a protocol so that when one process is in its critical section, **no other process** can enter its critical section for the same shared resource.

Structure:

```
do {
    [Entry Section]      // Request permission

        CRITICAL SECTION // Access shared data
```

```
    [Exit Section]       // Release permission

    REMAINDER SECTION    // Non-critical code
} while (true);
```

Q2. Three requirements for critical section solution

★★★[12-13, 13-14, 18-19 FINAL]

1. Mutual Exclusion

At most **one process** can be in the critical section at any time. If process Pi is in CS, no other process can enter.

2. Progress

If no process is in CS and some processes wish to enter, the selection of **who enters next** cannot be postponed indefinitely. Only processes **not in remainder section** participate in the decision.

3. Bounded Waiting

There exists a **limit/bound** on the number of times other processes can enter CS after a process requests entry and before that request is granted. Prevents **starvation**.

Q3. What are mutex locks? What is spinlock?

★★★[17-18, 18-19, 19-20, 20-21 FINAL]

Mutex Lock (Mutual Exclusion Lock)

Simplest synchronization tool. A boolean variable indicating resource availability.

Structure:

```
acquire() {
    while (!available)
        ; /* busy wait */
    available = false;
}

release() {
    available = true;
}
```

Usage:

```
acquire();
    // Critical Section
release();
```

Properties: - Atomic operations (hardware support) - Binary: locked (0) or unlocked (1) - Process must call `release()` — same process that acquired

Spinlock

A mutex lock where the process **spins** (busy-waits) in a loop while waiting.

Characteristics: - No context switch → efficient for **short waits** - **Wastes CPU cycles** during wait - Useful in **multiprocessor systems** (one CPU spins while another releases) - Not suitable for single-CPU or long waits

Advantage: No context-switch overhead
Disadvantage: CPU wastage during busy-waiting

Q4. Define liveness. Effect of liveness failure

★★★[17-18, 18-19, 19-20, 20-21 FINAL]

Liveness

Liveness refers to properties ensuring that a system makes **progress** — processes eventually make forward movement and complete execution.

A system satisfies liveness if: - Processes waiting for resources eventually obtain them - Processes in ready queue eventually execute - Blocked processes eventually resume

Liveness Failure — Two Forms:

1. Deadlock

Processes are waiting for each other in a **circular chain**. No process can proceed.

Example: P1 holds R1, waits for R2; P2 holds R2, waits for R1.

Effect: System **completely halted** for involved processes. Requires external intervention (restart/kill).

2. Starvation (Indefinite Blocking)

A process waits **indefinitely** because other processes continuously get preference.

Example: In priority scheduling, low-priority process never executes if high-priority processes keep arriving.

Effect: - Process never progresses despite being ready - Unfair resource allocation - Violates **bounded waiting** requirement - System appears functional but some processes never complete

---

Q5. Peterson's Solution — Explanation and Limitations

★★★[12-13, 13-14, 16-17, 17-18, 18-19, 20-21 FINAL]

Peterson's Solution (Software-based, 2 processes)

Shared Variables:

```
int turn;           // Whose turn to enter CS
boolean flag[2];    // flag[i]=true → Pi wants to enter CS
```

Process Pi:

```
do {
    flag[i] = true;         // I want to enter
    turn = j;               // Give turn to other
    while (flag[j] && turn == j)
        ;  // Busy wait if Pj wants AND it's Pj's turn

        CRITICAL SECTION

    flag[i] = false;        // I'm done

    REMAINDER SECTION
} while (true);
```

How it works: - Both processes set their flag=true - Both set turn to the other process - **Last writer** of `turn` determines who waits - If both want CS, one waits; if only one wants, it enters immediately

Satisfies: ✓ Mutual Exclusion
✓ Progress
✓ Bounded Waiting

Limitations

1. Modern Architectures

Not guaranteed on modern processors due to: - **Instruction reordering** by compilers/CPUs for optimization - May reorder `flag[i]=true` and `turn=j` - **No memory barriers** in basic implementation

2. Busy Waiting

Wastes CPU cycles in the while loop.

3. Only Two Processes

Does not generalize easily to N processes.

4. No Hardware Support

Relies on **load/store** being atomic, which isn't always guaranteed without special instructions.

Solution: Modern systems use hardware atomic instructions (test-and-set, compare-and-swap) with memory barriers.

Q6. Semaphore vs Mutex Lock. Limitations of Semaphores

★★★ [16-17, 19-20, 20-21 FINAL]

Comparison Table

Feature	Mutex Lock	Semaphore
Type	Binary (0 or 1)	Integer (0 to N)
Purpose	Mutual exclusion only	Synchronization + counting
Ownership	Process that locks must unlock	Any process can signal
Value	Locked/Unlocked	Count of available resources
Use Case	Protect critical section	Resource counting, signaling

Mutex Lock

```
acquire();
    // Critical Section
release();
```

- Must be released by the **same process** that acquired it
- Binary state

Semaphore

```
wait(S) {           // Also called P(), down()
    while (S <= 0)
        ; // busy wait
    S--;
}

signal(S) {         // Also called V(), up()
    S++;
}
```

Types: 1. **Binary Semaphore:** S ∈ {0,1} — similar to mutex 2. **Counting Semaphore:** S ∈ {0,1,2,…,N} — tracks N resources

Usage Example (N resources):

```
wait(semaphore);    // Decrement count, wait if 0
    // Use resource
signal(semaphore);  // Increment count
```

Limitations of Semaphores

1. Programming Errors

Easy to misuse:

```
signal(S);    // Wrong order
wait(S);
    // CS
signal(S);    // Violates mutual exclusion!
```

or

```
wait(S);
    // CS
wait(S);      // Forgot signal → deadlock!
```

2. No Ownership Tracking

Any process can call `signal()` even if it didn't call `wait()` → **inconsistency**.

3. Busy Waiting (Basic Implementation)

`while (S <= 0);` wastes CPU.
Solution: Use blocking semaphores with wait queues.

4. Deadlock Potential

Two processes:

```
P0: wait(S); wait(Q);    P1: wait(Q); wait(S);
```

→ Circular wait → **deadlock**

5. Difficult Debugging

Synchronization bugs are **non-deterministic** — timing-dependent, hard to reproduce.

CHAPTER 7: SYNCHRONIZATION EXAMPLES

Q7. Dining-Philosophers Problem + Solutions

★★★ [18-19, 19-20, 20-21 FINAL]

Problem Description

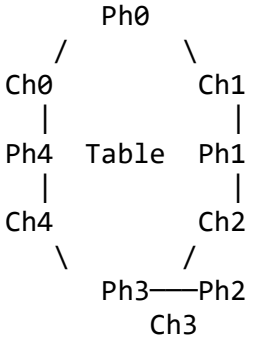
Setup: 5 philosophers sit at a round table. Between each pair is **one chopstick** (5 chopsticks total).

Philosopher behavior:

```
while (true) {
    think();
    pick up chopsticks;
    eat();
    put down chopsticks;
}
```

Rules: - To eat, philosopher needs **both** adjacent chopsticks (left AND right) - Can only pick up one chopstick at a time - After eating, puts down both chopsticks

Challenges: - Mutual exclusion (chopsticks are shared) - Avoid **deadlock** (all grab left chopstick → all wait for right) - Avoid **starvation** (one philosopher never eats)



Solution 1: Semaphore Solution

Shared Variables:

```
semaphore chopstick[5];  // All initialized to 1
```

Philosopher i:

```
do {
    wait(chopstick[i]);           // Pick up left
    wait(chopstick[(i+1) % 5]);   // Pick up right

        // Eat

    signal(chopstick[i]);         // Put down left
    signal(chopstick[(i+1) % 5]); // Put down right

        // Think
} while (true);
```

Problem: DEADLOCK! If all 5 philosophers pick up left chopstick simultaneously, all wait forever for right.

Deadlock-Free Semaphore Solutions:

Option A: Maximum 4 philosophers at table

```
semaphore room = 4;  // At most 4 can try to eat

wait(room);
    wait(chopstick[i]);
    wait(chopstick[(i+1)%5]);
        // Eat
    signal(chopstick[i]);
    signal(chopstick[(i+1)%5]);
signal(room);
```

Option B: Asymmetric (odd-even)

```
if (i % 2 == 0) {  // Even: pick left first
    wait(chopstick[i]);
    wait(chopstick[(i+1)%5]);
} else {            // Odd: pick right first
    wait(chopstick[(i+1)%5]);
    wait(chopstick[i]);
}
```

Breaks **circular wait** condition.

Option C: Pick both atomically

```
semaphore mutex = 1;

wait(mutex);  // Only one can pick chopsticks at a time
    wait(chopstick[i]);
    wait(chopstick[(i+1)%5]);
signal(mutex);
    // Eat
wait(chopstick[i]);
signal(chopstick[(i+1)%5]);
```

Solution 2: Monitor Solution

Monitor DiningPhilosophers:

```
enum {THINKING, HUNGRY, EATING} state[5];
condition self[5];  // Condition variable for each philosopher

void pickup(int i) {
    state[i] = HUNGRY;
    test(i);              // Try to acquire chopsticks
    if (state[i] != EATING)
```

```
        self[i].wait();   // Block if can't eat
}

void putdown(int i) {
    state[i] = THINKING;
    test((i+4) % 5);      // Check left neighbor
    test((i+1) % 5);      // Check right neighbor
}

void test(int i) {
    if (state[i] == HUNGRY &&
        state[(i+4)%5] != EATING &&  // Left not eating
        state[(i+1)%5] != EATING) {  // Right not eating

        state[i] = EATING;
        self[i].signal();  // Wake up if waiting
    }
}
```

Philosopher i calls:

```
DiningPhilosophers.pickup(i);
    // Eat
DiningPhilosophers.putdown(i);
```

Advantages: - No deadlock (controlled by test()) - No busy waiting - Clean abstraction

---

CHAPTER 8: DEADLOCKS

Q8. Four Necessary Conditions for Deadlock

★★★[12-13, 13-14, 18-19, 19-20, 20-21 FINAL]

Deadlock can occur **if and only if** all four conditions hold **simultaneously**:

1. Mutual Exclusion

At least one resource must be held in a **non-shareable mode**. Only one process at a time can use the resource.

Example: Printer, mutex lock

2. Hold and Wait

A process must be **holding at least one resource** and **waiting** to acquire additional resources held by other processes.

Example: P1 holds R1, waits for R2

3. No Preemption

Resources **cannot be forcibly taken** from a process. A resource can only be released **voluntarily** by the process holding it.

Example: Can't forcibly take a locked mutex from a process

4. Circular Wait

A **circular chain** of processes exists: P0 → P1 → … → Pn → P0, where each Pi waits for a resource held by Pi+1.

```
P1 holds R1, waits for R2
P2 holds R2, waits for R3
P3 holds R3, waits for R1  ← Circle!
```

All four must be present for deadlock. Breaking **any one** prevents deadlock.

Q9. System Model. Define Deadlock with Example

★★★ [17-18, 18-19, 19-20, 20-21 FINAL]

System Model

A system consists of: - **Resources:** R = {R1, R2, …, Rm} - **Processes:** P = {P1, P2, …, Pn}

Resource Types: - Each resource type Ri has Wi **instances** - Example: R1 = CPU (2 cores), R2 = Printer (3 units)

Process Resource Usage:

```
Request → Use → Release
```

Deadlock Definition

Deadlock is a situation where a set of processes is **permanently blocked**, each waiting for a resource held by another process in the set. No process can proceed.

Multithreaded Application Example

Scenario: Two threads, two mutex locks

```
pthread_mutex_t lock1, lock2;

// Thread 1:
pthread_mutex_lock(&lock1);
    pthread_mutex_lock(&lock2);  // Waits if T2 holds lock2
        // Critical Section
    pthread_mutex_unlock(&lock2);
```

```
pthread_mutex_unlock(&lock1);

// Thread 2:
pthread_mutex_lock(&lock2);
    pthread_mutex_lock(&lock1);  // Waits if T1 holds lock1
        // Critical Section
    pthread_mutex_unlock(&lock1);
pthread_mutex_unlock(&lock2);
```

Deadlock Scenario:

```
Time 1: T1 acquires lock1
Time 2: T2 acquires lock2
Time 3: T1 tries to acquire lock2 → BLOCKS (T2 holds it)
Time 4: T2 tries to acquire lock1 → BLOCKS (T1 holds it)
        → DEADLOCK!
```

All 4 conditions satisfied: 1. Mutual exclusion: locks are non-shareable ✓ 2. Hold and wait: T1 holds lock1, waits for lock2 ✓ 3. No preemption: can't forcibly take locks ✓ 4. Circular wait: T1→lock2→T2→lock1→T1 ✓

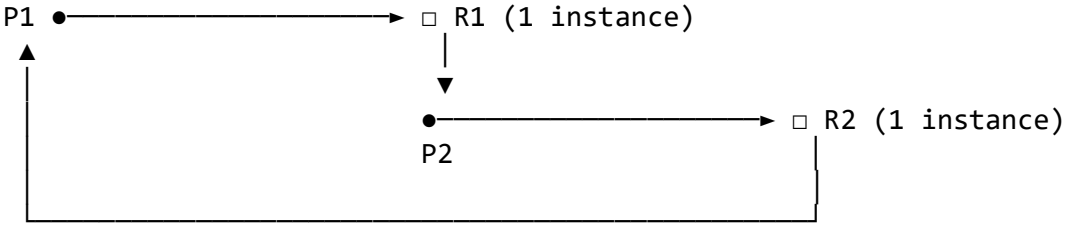
Q10. Resource-Allocation Graphs

★★★ [17-18, 18-19, 19-20, 20-21 FINAL]

Graph Components

- **Process:** Circle (Pi)
- **Resource Type:** Rectangle (Rj)
- **Resource Instance:** Dot inside rectangle
- **Request Edge:** Pi → Rj (process requests resource)
- **Assignment Edge:** Rj → Pi (resource allocated to process)

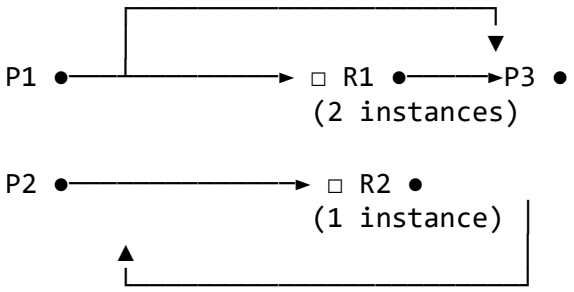
Example 1: Graph with Deadlock



Analysis: - P1 holds R1, requests R2 - P2 holds R2, requests R1 - **Cycle exists:** P1→R2→P2→R1→P1 - Each resource has **1 instance** - **DEADLOCK!**

Rule: If graph has a cycle AND each resource type has **only 1 instance** → **Deadlock exists**

Example 2: Cycle BUT No Deadlock



Better ASCII representation:

```
P1 requests R1
R1 (2 instances) → one to P1, one to P3
P2 holds R2
P3 requests R2
P2 requests R1
```

Graph: - Cycle: P1→R1→P3→R2→P2→R1→P1 ✓ - BUT R1 has **2 instances** - P3 can finish (already has R1), release R1 - Then P2 can get R1, finish - **NO DEADLOCK** (cycle is not sufficient when multiple instances exist)

Rule: Cycle is **necessary but not sufficient** for deadlock when resources have multiple instances.

Summary

Graph Property	Single Instance per Resource	Multiple Instances
No Cycle	No deadlock	No deadlock
Cycle Exists	**DEADLOCK**	**Maybe** deadlock

To confirm deadlock with multiple instances → need **deadlock detection algorithm**.

Q11. Banker's Algorithm — Data Structures

★★★ [17-18, 18-19 FINAL]

Purpose

Deadlock avoidance — ensures system **never enters** an unsafe state by checking requests before granting.

Data Structures (n processes, m resource types)

1. Available [m]

Number of **available instances** of each resource type.
`Available[j] = k` → k instances of Rj are available

2. Max [n][m]

Maximum demand of each process.
`Max[i][j] = k` → Pi may request at most k instances of Rj

3. Allocation [n][m]

Resources **currently allocated** to each process.
`Allocation[i][j] = k` → Pi currently holds k instances of Rj

4. Need [n][m]

Remaining resource need for each process.
`Need[i][j] = Max[i][j] - Allocation[i][j]`

Relationship:

```
Need = Max - Allocation
Available = Total - Σ Allocation
```

Safety Algorithm (checks if state is safe)

Step 1: Initialize

```
Work = Available
Finish[i] = false  for all i
```

Step 2: Find process i such that:

```
Finish[i] == false  AND  Need[i] ≤ Work
```

If found, go to Step 3. If none, go to Step 4.

Step 3: Simulate process i completing:

```
Work = Work + Allocation[i]
Finish[i] = true
Go back to Step 2
```

Step 4: Check result:

```
If Finish[i] == true for ALL i → SAFE state
Else → UNSAFE state
```

Example (Book Standard)

System: 3 resource types (A, B, C), 5 processes

Process	Allocation	Max	Need

Process	Allocation	Max	Need
	A B C	A B C	A B C
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Total resources: A=10, B=5, C=7
Available: (10-7, 5-2, 7-5) = **(3, 3, 2)**

Safety Check:

Step	Find Process	Need ≤ Work?	Work After
1	P1	(1,2,2) ≤ (3,3,2)? YES	(3,3,2)+(2,0,0) = (5,3,2)
2	P3	(0,1,1) ≤ (5,3,2)? YES	(5,3,2)+(2,1,1) = (7,4,3)
3	P4	(4,3,1) ≤ (7,4,3)? YES	(7,4,3)+(0,0,2) = (7,4,5)
4	P0	(7,4,3) ≤ (7,4,5)? YES	(7,4,5)+(0,1,0) = (7,5,5)
5	P2	(6,0,0) ≤ (7,5,5)? YES	(7,5,5)+(3,0,2) = (10,5,7)

Safe sequence: <P1, P3, P4, P0, P2>
State is SAFE → No deadlock possible

Side Effects of Deadlock Prevention

Methods: Break one of the four conditions

1. Deny Mutual Exclusion
Side effect: Impossible for non-shareable resources (printers, locks)

2. Deny Hold and Wait (require all resources at once)
Side effects: - **Low resource utilization** (hold resources even when not using) - **Starvation** possible (process needing many resources may never get all)

3. Allow Preemption
Side effects: - **Not practical** for many resources (can't preempt a printer mid-job) - **Data loss** or inconsistency - **Complexity** in saving/restoring state

4. Deny Circular Wait (order resources)
Side effects: - **Inconvenient** for programmers - **Reduced concurrency** - May conflict with application logic

---

Q12. Deadlock Detection Algorithm

★★★ [18-19, 19-20, 20-21 FINAL]

Data Structures

Same as Banker's Algorithm, but **no Max** matrix:

1. **Available[m]:** Available instances of each resource
2. **Allocation[n][m]:** Currently allocated resources
3. **Request[n][m]:** Current resource requests (not future max)

Note: Request[i] = 0 means Pi is not requesting anything currently.

Detection Algorithm

Step 1: Initialize

```
Work = Available
Finish[i] = false  for all i

// Mark processes with no allocation as finished
For i = 1 to n:
    if Allocation[i] == 0:
        Finish[i] = true
```

Step 2: Find index i such that:

```
Finish[i] == false  AND  Request[i] ≤ Work
```

If found → Step 3. Else → Step 4.

Step 3:

```
Work = Work + Allocation[i]
Finish[i] = true
Go to Step 2
```

Step 4:

```
If Finish[i] == false for some i:
    DEADLOCK exists (processes with Finish[i]=false are deadlocked)
Else:
    No deadlock
```

Example (Multiple Instances)

5 processes, 3 resource types (A=7, B=2, C=6)

Allocation: | | A | B | C | |—–|—|—|—| | P0 | 0 | 1 | 0 | | P1 | 2 | 0 | 0 | | P2 | 3 | 0 | 3 | | P3 | 2 | 1 | 1 | | P4 | 0 | 0 | 2 |

Request: | | A | B | C | |—–|—|—|—| | P0 | 0 | 0 | 0 | | P1 | 2 | 0 | 2 | | P2 | 0 | 0 | 0 | | P3 | 1 | 0 | 0 | | P4 | 0 | 0 | 2 |

Available: (7-7, 2-2, 6-6) = (0, 0, 0)

Detection Steps:

Step	Work	Process	Request≤Work?	Action	New Work
1	(0,0,0)	P0	(0,0,0)≤(0,0,0)? YES	Finish[0]=T	(0,1,0)
2	(0,1,0)	P2	(0,0,0)≤(0,1,0)? YES	Finish[2]=T	(3,1,3)
3	(3,1,3)	P3	(1,0,0)≤(3,1,3)? YES	Finish[3]=T	(5,2,4)
4	(5,2,4)	P1	(2,0,2)≤(5,2,4)? YES	Finish[1]=T	(7,2,4)
5	(7,2,4)	P4	(0,0,2)≤(7,2,4)? YES	Finish[4]=T	(7,2,6)

All Finish[i] = true → NO DEADLOCK

Example Showing Deadlock

If **P2 requests (0,0,1)** additional:

Request matrix becomes: | | A | B | C | |—–|—|—|—| | P2 | 0 | 0 | **1** |

Now P2's request (0,0,1) > Work (0,0,0) → cannot grant.

After P0 finishes: Work = (0,1,0)
P2 request (0,0,1) > (0,1,0)? Still NO.
No other process can proceed → **DEADLOCK**

Deadlocked processes: P1, P2, P3, P4

CHAPTER 9: MAIN MEMORY

Q13. Logical vs Physical Address. MMU Function

★★★ [18-19, 19-20, 20-21 FINAL]

Logical Address (Virtual Address)

- Generated by **CPU** during program execution
- What the **program sees**
- Address space: **0 to max** (e.g., 0 to 2^32-1 for 32-bit)

- **Independent** of physical memory layout

Physical Address

- **Actual address** in main memory (RAM)
- What the **memory unit sees**
- Corresponds to real hardware locations

Key Difference

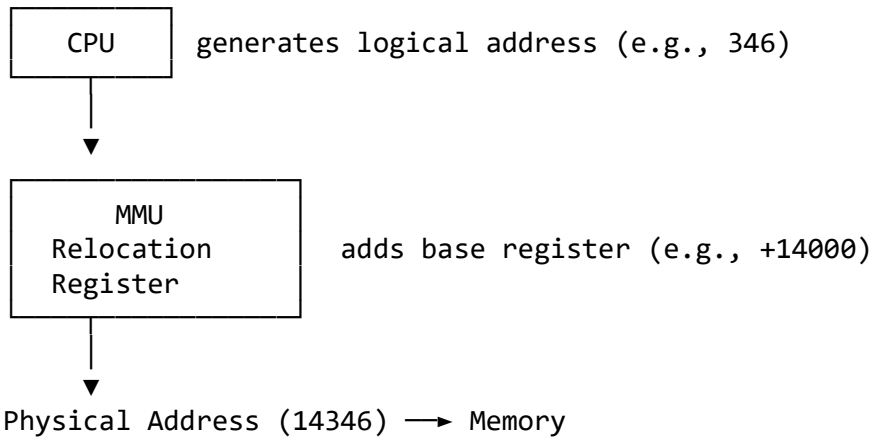
```
Logical Address  ──────────►  Physical Address
  (CPU uses)      [via MMU]    (RAM uses)
```

Compile/Load time: Logical = Physical (absolute addresses)
Execution time: Logical ≠ Physical (mapping needed)

Memory Management Unit (MMU)

MMU = Hardware device that **translates** logical addresses to physical addresses at runtime.

Basic Relocation MMU:



Formula:

```
Physical Address = Logical Address + Relocation Register
```

Functions: 1. **Address Translation:** Logical → Physical 2. **Memory Protection:** Check if address is within bounds 3. **Relocation Support:** Allow programs to load anywhere in memory

Example: - Process allocated memory starting at 14000 - CPU generates logical address 346 - MMU: 346 + 14000 = **14346** (physical address) - Memory fetches data from location 14346

Q14. Paging — Explanation and Illustration

★★★[18-19, 19-20, 20-21 FINAL]

What is Paging?

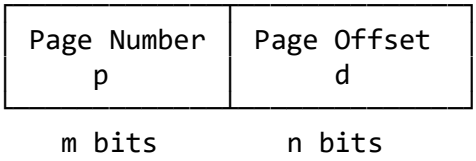
Paging is a memory-management scheme that allows a process's **physical address space** to be **noncontiguous**. Eliminates external fragmentation.

Key Concepts: - **Frame:** Fixed-size block of **physical** memory (e.g., 4KB) - **Page:** Fixed-size block of **logical** memory (same size as frame) - **Page Table:** Maps logical pages to physical frames

Page Size = Frame Size (typically 4KB, 8KB, or 4MB)

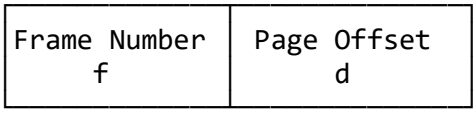
Address Translation

Logical Address (generated by CPU):



Translation: 1. CPU generates logical address (p, d) 2. Page number **p** → index into **page table** 3. Page table gives **frame number f** 4. Physical address = (f, d)

Physical Address:



Example: 32-byte Memory, 4-byte Pages

Physical Memory:

```
Frame 0: [0-3]     ┐
Frame 1: [4-7]     │
Frame 2: [8-11]    ├─ 8 frames
Frame 3: [12-15]   │  (32 bytes / 4 bytes)
Frame 4: [16-19]   │
Frame 5: [20-23]   │
Frame 6: [24-27]   │
Frame 7: [28-31]   ┘
```

Logical Address Space (16 bytes):

```
Page 0: [0-3]   ┐
Page 1: [4-7]   ├─ 4 pages
Page 2: [8-11]  │  (16 bytes / 4 bytes)
Page 3: [12-15] ┘
```

Page Table for Process: | Page | Frame | |——|———-| | 0 | 5 | | 1 | 2 | | 2 | 7 | | 3 | 1 |

Translation Example:

Logical Address 5: - 5 in binary: 0101 - Page bits (2 MSBs): **01** → Page 1 - Offset bits (2 LSBs): **01** → Offset 1 - Page table: Page 1 → Frame 2 - Physical address: Frame 2, Offset 1 → **2×4 + 1 = 9**

Verification:

```
Logical:  Page 1, byte 1 (second byte of page 1)
Physical: Frame 2 is at addresses [8-11]
          Offset 1 → address 9 ✓
```

Memory Layout:

```
Logical Memory:      Physical Memory:
Page 0 [0-3]    →    Frame 5 [20-23]
Page 1 [4-7]    →    Frame 2 [8-11]
Page 2 [8-11]   →    Frame 7 [28-31]
Page 3 [12-15]  →    Frame 1 [4-7]
```

Advantages: - No external fragmentation - Easy allocation (any free frame) - Sharing possible (point multiple pages to same frame)

Disadvantage: - Internal fragmentation (average 0.5 × page size per process)

Q15. Memory Allocation: First-Fit, Best-Fit, Worst-Fit

★★★[11-12, 12-13, 13-14 FINAL] — *Book Exercise 9.6, 9.13*

Given Data

Memory Partitions (in order): 100 KB, 500 KB, 150 KB, 350 KB, 600 KB

Process Requests (in order): 110 KB, 419 KB, 210 KB, 423 KB

Algorithm 1: First-Fit

Rule: Allocate the **first** partition that is large enough.

Request	Scan Partitions	Decision	Remaining
110 KB	100(no), 500(**yes**)	→ 500 KB	390 KB remains

Request	Scan Partitions	Decision	Remaining
419 KB	100(no), 390(no), 150(no), 350(no), 600(**yes**)	→ 600 KB	181 KB remains
210 KB	100(no), 390(**yes**)	→ 390 KB	180 KB remains
423 KB	100(no), 180(no), 150(no), 350(no), 181(no)	**CANNOT ALLOCATE**	—

Result: - Allocated: 110, 419, 210 - **Failed:** 423 KB - **Not efficient**

Algorithm 2: Best-Fit

Rule: Allocate the **smallest** partition that is large enough.

Request	Partitions ≥ Size	Smallest	Decision	Remaining
110 KB	500, 150, 350, 600	150 KB	→ 150 KB	40 KB
419 KB	500, 350, 600	500 KB	→ 500 KB	81 KB
210 KB	350, 600	350 KB	→ 350 KB	140 KB
423 KB	600	600 KB	→ 600 KB	177 KB

Result: - **All 4 processes allocated successfully!** - Remaining: 100 KB, 81 KB, 40 KB, 140 KB, 177 KB

Algorithm 3: Worst-Fit

Rule: Allocate the **largest** partition.

Request	Largest Available	Decision	Remaining
110 KB	600 KB	→ 600 KB	490 KB
419 KB	500 KB	→ 500 KB	81 KB
210 KB	490 KB	→ 490 KB	280 KB
423 KB	350 KB (too small!)	**CANNOT ALLOCATE**	—

Result: - Allocated: 110, 419, 210 - **Failed:** 423 KB

Comparison Summary

Algorithm	Allocated	Failed	Efficiency
First-Fit	3 of 4	423 KB	Poor
Best-Fit	**4 of 4**	**None**	**BEST**
Worst-Fit	3 of 4	423 KB	Poor

Answer: Best-Fit makes the most efficient use of memory because it allocated all four processes successfully.

Reason: Best-fit minimizes wasted space by choosing the tightest fit, leaving larger blocks available for future large requests.

QUICK REFERENCE FORMULAS

```
Turnaround Time = Completion - Arrival
Waiting Time = Turnaround - Burst
Response Time = First Start - Arrival

Logical Address = (Page Number, Offset)
Physical Address = (Frame Number, Offset)

Page Number = Logical Address / Page Size
Offset = Logical Address % Page Size

EAT (with TLB):
  = Hit_Ratio × (TLB_time + Mem_time)
  + (1 - Hit_Ratio) × (TLB_time + 2×Mem_time)

Number of Pages = ⌈Process Size / Page Size⌉
Internal Fragmentation ≈ 0.5 × Page Size (average)
```

End of ★★★Answers — Muhtasim Sir's Part (Chapters 6–9)
Reference: Operating System Concepts, 10th Edition